arXiv:2603.01549v1 [cs.CV] 2 Mar 2026

Pri4R: Learning World Dynamics for Vision-Language-Action Models with Privileged 4D Representation

Jisoo Kim*[1,2], Jungbin Cho*[3,5], Sanghyeok Chu[2,4], Ananya Bal[5], Jinhyung Kim[2], Gunhee Lee[2], Sihaeng Lee[2], Seung Hwan Kim[2] Bohyung Han†[4], Hyunmin Lee†[2], Laszlo A. Jeni†[5], Seungryong Kim†[1]
[1]KAIST AI [2]LG AI Research, [3]Yonsei University, [4]Seoul National University, [5]Carnegie Mellon University
https://jiiiisoo.github.io/Pri4R/

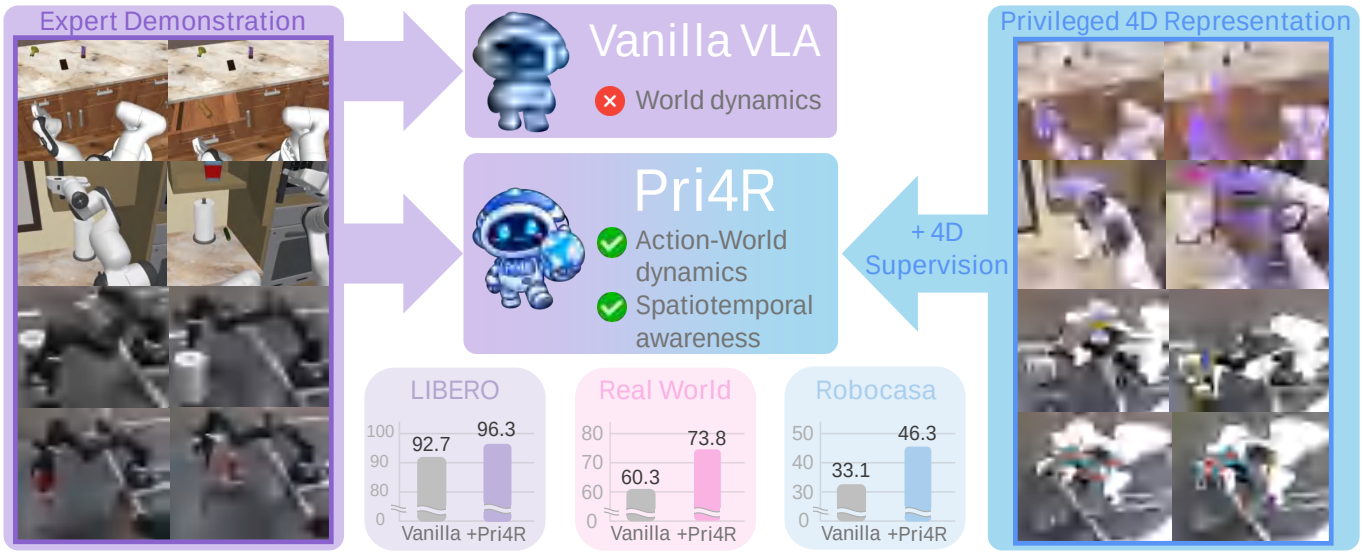

Fig. 1: **Pri4R** equips Vision–Language–Action (VLA) models with an implicit awareness of action–world dynamics via *privileged* 4D geometric supervision. Unlike standard VLAs (top) trained only by action imitation (left), Pri4R extracts 3D point tracks from demonstrations (right) and adds an auxiliary head to predict future point trajectories alongside actions. This training signal encourages the policy to model how scene geometry evolves under interaction, improving robustness and task success (bottom), while preserving the original test-time interface with zero inference overhead.

***Abstract*—Humans learn not only how their bodies move, but also how the surrounding world responds to their actions. In contrast, while recent Vision-Language-Action (VLA) models exhibit impressive semantic understanding, they often fail to capture the spatiotemporal dynamics governing physical interaction. In this paper, we introduce Pri4R, a simple yet effective approach that endows VLA models with an implicit understanding of world dynamics by leveraging privileged 4D information during training. Specifically, Pri4R augments VLAs with a lightweight point track head that predicts 3D point tracks. By injecting VLA features into this head to jointly predict future 3D trajectories, the model learns to incorporate the evolving scene geometry within its shared representation space. This allows the action prediction components to leverage a more physically-aware context for precise control. Due to its architectural simplicity, Pri4R is seamlessly compatible with dominant VLA design patterns through minimal changes. During inference, we run the model using the original VLA architecture unchanged; Pri4R adds no extra inputs, outputs, or computational overhead during inference. Across simulation and real-world evaluations, Pri4R significantly improves performance on challenging manipulation tasks, including a +10% gain on LIBERO-Long and a +40% gain on RoboCasa. We further show that 3D point track prediction is an effective supervision target for learning action-world dynamics, while also validating our design choices through extensive ablations. Code and checkpoints will be released.**

I. Introduction

Generalizable robot policies require not only semantic understanding of the surrounding environment, but also knowledge of the dynamics governing action–world interactions. Recent advances in Vision–Language Models (VLMs) [2, 37], together with the emergence of large-scale robot datasets [27, 42, 54], have enabled Vision–Language–Action (VLA) models [7, 24, 28] to leverage pretrained VLMs for robotic control. A key design principle of these approaches is to minimize architectural changes, introducing only a lightweight action head to transfer the high-level semantic capabilities of VLMs

to robot manipulation.

However, this adaptation uses a sole learning signal: simple action labels, which primarily encourage *imitation of demonstrated motions*, with no knowledge about *the dynamics of the world*. **Put simply, action labels specify *how to move*, but not *what will happen*.** As a result, the learned policies often produce semantically plausible actions that lack an understanding of underlying world dynamics (e.g., attempting to grasp a handle without accounting for the door's kinematic constraints), leading to brittle or inaccurate object interactions and ultimately, task failure.

In contrast, humans reason about actions through an internal understanding of geometry and dynamics, anticipating *how their embodiment and surrounding objects will move, deform, or make contact in response to actions* [15, 19, 25, 39, 58]. Some efforts sought to incorporate such predictive capabilities by introducing additional models that generate future images or states [5, 16, 21, 59]. More recent approaches instead train a uniform policy to predict actions together with additional signals, including high-level abstractions (e.g., language [48, 69], feature embeddings [10, 65], object-centric representations [24, 66]), and even dense observations (e.g., images [51, 66] and depth maps [65]).

Although these signals provide extra knowledge for action prediction, they are not directly aligned with the *spatiotemporal metric space* in which action–world interactions unfold, yielding only indirect supervision where the model must still learn to utilize these cues for precise control. This mismatch suggests that truly modeling world dynamics requires a representation that is both metric and temporally grounded, namely *how 3D geometry evolves over time (4D)*, rather than predicting static abstractions or goal observations. Furthermore, the aforementioned methods require additional computation during inference, along with complex design choices and hyperparameters.

In this paper, we propose **Pri4R**, a simple yet effective framework that equips VLA models with an implicit knowledge of world dynamics, enabling more accurate and robust behavior. The key idea is to utilize 4D geometry as a privileged supervision signal during training to revise the VLM's latent representations. By injecting the VLM's internal embeddings into a dedicated point tracking head, we guide the backbone to encode the causal relationship between actions and the resulting geometric evolution of the scene. This shared, dynamics-aware representation, then directly empowers the action prediction head to reason with a deeper physical context, without introducing additional inputs or architectural changes at inference time. Figure 1 illustrates the concept of Pri4R.

Specifically, we first precompute 3D point tracks for the demonstration trajectories used for training. We then augment the VLA with a lightweight point tracking head that predicts future 3D trajectories, conditioned on the model's internal embeddings. This design ensures that the gradients from the point tracking task inform and enrich the shared feature space used for action prediction. Pri4R is broadly applicable and easy to integrate into existing VLA frameworks with minimal architectural changes, for both backbone-centric VLAs [28] and expert-style VLAs [7, 24]. At test time, the auxiliary units are discarded, allowing the original VLA architecture to run unchanged while benefiting from the integrated information about world dynamics.

Through extensive experiments on multi-task simulation benchmarks (LIBERO and RoboCasa) and real-world evaluations, we show that Pri4R consistently outperforms state-of-the-art baselines across all settings. We further analyze supervision target variants and find that 3D point track prediction most effectively encourages learning of action-world dynamics. Finally, we ablate the core design choices of Pri4R and show that each component is necessary for learning world dynamics and improving downstream performance.

Our contributions are summarized as follows:

- We propose Pri4R, which facilitates learning world dynamics by leveraging 3D point tracks as privileged supervision [18] during training to enrich the shared representation of VLA models.
- We show that Pri4R consistently improves the performance of SOTA VLA models across simulation and real-world tasks without any inference time overhead or architectural changes.
- Through systematic analysis, we demonstrate that predicting 3D point tracks is particularly effective for learning action-world dynamics and extensively ablate each design choice of Pri4R.

II. Related Work

Vision-Language-Action models. Pioneered by Robotics Transformers [8, 70], Vision–Language–Action (VLA) models [7, 10, 11, 22, 24, 28, 32, 34, 38, 50] adapt large pretrained Vision–Language Models (VLMs) [2, 13, 37] for robotic control by predicting actions conditioned on visual observations and language instructions, typically via action-specific heads or modules. OpenVLA [28] trains a 7B VLM [26] by autoregressively predicting discrete action tokens on the OpenX [42] dataset. Follow-up works improve OpenVLA by reducing the model footprint (e.g., to 1B parameters) [4] or by incorporating visual trace prompting [68]. In contrast, other methods [7, 33, 50, 57] use generative action heads (e.g., diffusion or flow matching) to produce continuous actions. More recently, OpenVLA-OFT [29] shows that L1 regression on continuous actions with parallel decoding can both improve performance and substantially increase inference throughput.

Improving VLAs with forecasting. Inspired by human anticipatory reasoning, forecasting has played an important role in VLAs. One line of work [1, 5, 9, 16, 21, 59] employs generative models to explicitly predict future states, images, or videos, and then conditions action policies on these predicted modalities. This can be seen as the generative model serving as a high-level planner, guiding the low-level policy to improve task achievement. However, these approaches are often constrained by the accuracy of the predicted signals and the additional inference-time latency they introduce. In contrast, more recent methods leverage the VLA itself to

produce auxiliary signals within the same model, using them as intermediate reasoning either before or alongside action prediction at test time. Concretely, this is achieved either by first generating intermediate signals and then causally predicting actions [10, 24, 51, 66, 67, 69], or by generating signals and actions jointly in parallel [46, 65]. Although effective, these auxiliary signals (e.g., language, feature embeddings, images, videos) are not expressed in the same spatiotemporal metric space as actions, and therefore provide only indirect supervision for action learning.

Point tracking for manipulation. 2D point tracks have been actively used in robot learning [62, 64, 68], but they provide limited geometric information. With the emergence of strong 3D point tracking models [17, 61], recent work has begun to adopt 3D point tracks for robotic manipulation as a structured spatiotemporal representation for downstream learning and control. In particular, point tracks have been used for policy learning [45, 56, 63], reward modeling [43, 47, 62], and even as action representations [23, 41]. In our work, we use 3D point tracks as a privileged training signal to encourage the model to learn geometric 4D world dynamics that are directly relevant for action prediction, without requiring point tracks at inference time.

III. Preliminaries

This section formalizes the Vision-Language-Action (VLA) framework and reviews representative baselines, providing the background to discuss how Pri4R incorporates world dynamics into these formulations.

A. VLA Formulation

Vision-Language-Action (VLA) models are typically developed by fine-tuning pretrained VLMs via imitation learning [3]. At each time step t, the policy receives an observation $\mathbf{o}_t \triangleq (\mathbf{I}_t, \mathbf{t}_t, \mathbf{q}_t)$ and predicts an action chunk $\mathbf{a}_{t:t+H}$ over a horizon H, where $\mathbf{I}_t = \{\mathbf{I}_t^i\}_{i=1}^N$ denotes a set of N multi-view camera images, $\mathbf{t}_t$ is a tokenized language instruction, and $\mathbf{q}_t$ is the robot proprioceptive state. Note that, if $H = 1$, the chunk reduces to a single action $\mathbf{a}_t$.

In this framework, the images $\mathbf{I}_t$ and robot state $\mathbf{q}_t$ are encoded by modality-specific encoders and projected onto the backbone's token embedding space alongside $\mathbf{t}_t$. Given these inputs, the VLM backbone produces high-level multi-modal embeddings. Since standard VLMs are not designed to output action chunks, VLAs attach an additional *action-prediction component* (e.g., a lightweight action head) that consumes these embeddings to parameterize an action distribution conditioned on the backbone's internal representations (and optionally $\mathbf{q}_t$). The full set of model parameters θ is trained end-to-end on a demonstration dataset $\mathcal{D}$ with behavior cloning [44], maximizing the log-likelihood of the demonstrated action chunks, which is given by

$$\max_{\theta} \mathbb{E}_{(\mathbf{a}_{t:t+H}, \mathbf{o}_t)\sim\mathcal{D}} \log p_\theta(\mathbf{a}_{t:t+H} \mid \mathbf{o}_t). \tag{1}$$

B. VLA Baselines

To adopt our approach, we consider two state-of-the-art VLA architectures with distinct design patterns: OpenVLA-OFT [29] and the π series [7, 24].

OpenVLA-OFT employs an Optimized Fine-Tuning (OFT) recipe that enhances the original OpenVLA through (i) parallel decoding with bidirectional attention, (ii) action chunking, (iii) a continuous action representation, and (iv) a simple ℓ_1 regression objective. Architecturally, OpenVLA-OFT replaces the discrete action-token output with an MLP regression head. This MLP head processes the final-layer hidden states at the action query token positions and maps them directly to continuous action chunks $\mathbf{a}_{t:t+H}$, enabling efficient, non-autoregressive prediction.

The π series, in contrast, generates action chunks through a flow-matching framework. It augments the pretrained VLM backbone with a dedicated transformer action expert. This expert takes the robot proprioceptive state $\mathbf{q}_t$ and a noisy action chunk as inputs, predicting a corresponding vector field (velocity) for iterative denoising. The action expert integrates with the backbone by attending to its internal hidden states via shared self-attention. To preserve the VLM's pretrained representations, π employs a blockwise causal attention mask; while the robotics-specific action tokens (including the noisy action chunk) can attend to the full multi-modal context, the language and image tokens are restricted from attending to these action-related tokens.

IV. Pri4R: Learning World Dynamics via Privileged 4D Representations

Pri4R is a framework that incorporates privileged geometric information to improve the world dynamics understanding of VLA models. During training, we use high-fidelity 4D signals as an auxiliary supervision to refine the internal representations of the VLM backbone. By supervising the model to predict the physical evolution of the scene, we enable the VLA to develop a physically-aware context for robot control, without requiring any additional inputs or computational overhead during inference.

A. Learning from Privileged Point Track Head

To implement this privileged learning strategy, we augment the VLM backbone with a lightweight *point track head* that predicts *future 3D trajectories* of scene points. By feeding the backbone's multi-modal embeddings into this head, the shared representations are forced to encode the spatiotemporal constraints of the environment. The point track head is deliberately lightweight, consisting of two small MLPs that interface with the backbone without altering its original architecture.

At time t, the *point MLP* embeds the current point set $P_t \in \mathbb{R}^{N_p\times 3}$ into per-point features $\mathbf{e}_t \in \mathbb{R}^{N_p\times d}$. To integrate scene context, we introduce a sequence of multi-modal embeddings over the action horizon, $\mathbf{z}_t = \phi(\mathbf{o}_t) \in \mathbb{R}^{H\times d}$, which is derived from the backbone ϕ and consumed by the action head. We broadcast $\mathbf{z}_t$ across points to obtain a tensor in $\mathbb{R}^{H\times N_p\times d}$ and concatenate it with the per-point features $\mathbf{e}_t$ (broadcast across the horizon H). The resulting tensor is fed into a *fusion MLP* to predict per-step 3D displacements:

$$\widehat{\Delta P}_{t:t+H} = \mathrm{MLP}_{\text{fusion}}(\mathbf{z}_t \oplus \mathbf{e}_t) \in \mathbb{R}^{H\times N_p\times 3}, \tag{2}$$

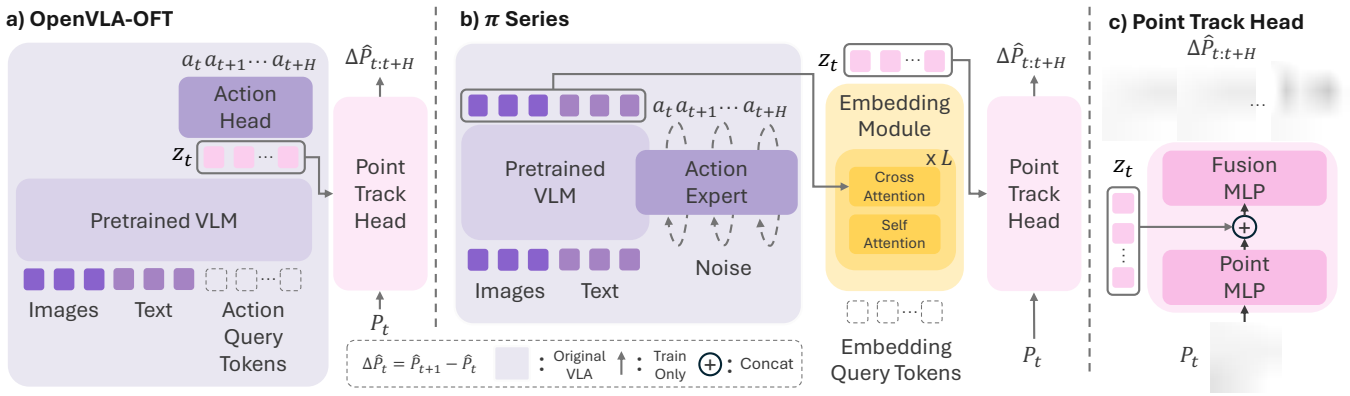

Fig. 2: **Overview of Pri4R.** We augment two common VLA architectures with an auxiliary point track head that predicts per-step 3D point displacements $\widehat{\Delta P}_{t:t+H}$ from backbone embeddings $\mathbf{z}_t$ and the current point set P_t. (a) For backbone-centric VLAs (e.g., OpenVLA-OFT [28]), we set $\mathbf{z}_t$ to the final layer action-query token embeddings. (b) For expert-style VLAs (e.g., π [7, 24]), we condition an embedding module on the backbone's final layer hidden states to produce $\mathbf{z}_t$. (c) The point track head encodes P_t with a PointMLP, then fuses the resulting point features with $\mathbf{z}_t$ via a FusionMLP to predict future point tracks. Privileged 3D point track supervision during training forces the VLA to model how scene geometry evolves, yielding more reliable interaction and higher task success, while leaving the test-time interface and compute completely unchanged.

where $\oplus$ denotes the feature concatenation operator.

Through this architecture, the auxiliary loss gradients from the point track head are backpropagated into the VLM backbone, encouraging it to refine its shared representation to capture essential world dynamics. By jointly optimizing for both action and 3D trajectory prediction, the VLA model understands a richer, geometry-aware context that is inherently grounded in the physical evolution of the scene. While the core prediction mechanism remains consistent, the interface between the point track head and the VLA backbone is tailored to the specific design of each model family. We examine the OpenVLA-OFT and π family models; their detailed designs are illustrated in Figure 2 and described below.

OpenVLA-OFT: We set $\mathbf{z}_t$ to the final-layer hidden states of the action query tokens (i.e., the empty action embeddings) produced by the backbone. Since these are the exact embeddings mapped to actions by the action head, injecting them into the point track head allows the backbone to encode the underlying scene dynamics necessary for precise control.

π family: Since the action expert in π models interacts with the backbone via masked self-attention rather than a fixed embedding, the choice of $\mathbf{z}_t$ is less straightforward. We introduce a lightweight transformer embedding module that takes a set of learnable query tokens and applies cross-attention over the final-layer image and language tokens from the VLM backbone. This yields an action-horizon embedding sequence $\mathbf{z}_t \in \mathbb{R}^{H \times d}$, which is then processed by the point track head identically to the OpenVLA-OFT case.

B. Why 3D Point Tracks as Privileged Supervision?

Although prior works have explored a variety of predictive representations to train VLAs [10, 24, 46, 51, 65–67, 69], these signals are often ill-suited for learning world dynamics. Traditional representations are frequently (1) *temporally sparse*, predicting only a goal observation at the end of the action horizon; (2) *lack explicit spatial structure*, as language or latent feature embeddings do not preserve metric geometry; and (3) *spatially redundant*, since dense predictions such as images or depth maps largely reproduce the input observation [65].

We therefore use *3D point tracks* as the target representation. Compared to other representations, 3D point tracks are: (1) *temporally dense*, matching the action horizon to capture fine-grained world interaction; (2) *geometric*, providing metric 3D structure that promotes spatial awareness; and (3) *spatially sparse*, enabling an efficient learning objective by focusing on a compact set of informative points rather than redundant dense grids like video or depth maps [65]. Most importantly, 3D point tracks reside in the same *spatiotemporal metric space* as robot actions, providing a supervisory signal that is naturally aligned with control.

C. Construction of 3D Point Track Supervision

To realize the proposed learning with a privileged information framework, we obtain 3D point tracks for every demonstration in $\mathcal{D}$ to serve as privileged supervision. For each training sample $(\mathbf{a}_{t:t+H}, \mathbf{o}_t) \sim \mathcal{D}$, we assume access to N_p tracked points with fixed identities over the same horizon and denote the point set at time τ by

$$P_\tau = \{p_j^\tau\}_{j=1}^{N_p}, \qquad p_j^\tau = (x_j^\tau, y_j^\tau, z_j^\tau) \in \mathbb{R}^3,$$

where $\tau \in \{t, \ldots, t+H+1\}$ and $j \in \{1, \ldots, N_p\}$. We refer to the sequence $\{P_\tau\}_{\tau=t}^{t+H+1}$ as the 3D point tracks for the demonstration window starting at time t.

Rather than directly regressing absolute positions, we supervise the point track head with per-step 3D displacements,

$$\Delta p_j^\tau = p_j^{\tau+1} - p_j^\tau, \qquad \tau \in \{t, \ldots, t+H\},$$

and denote $\Delta P_\tau = \{\Delta p_j^\tau\}_{j=1}^{N_p}$ and $\Delta P_{t:t+H} = \{\Delta P_\tau\}_{\tau=t}^{t+H}$. Given the backbone embeddings derived from the current observation $\mathbf{o}_t$ and the current point set P_t, the point track head

TABLE I: **Results on LIBERO.** Success rates (SRs) are reported across four task suites (500 trials per suite). Entries marked with * are taken from the OpenVLA-OFT paper [29]. Best performances for each task are marked in **bold**. Pri4R improves average success rates of every state-of-the-art VLAs (π series and OpenVLA-OFT) and almost every task in LIBERO.

	Average	LIBERO - Spatial	LIBERO - Object	LIBERO - Goal	LIBERO - Long
	SR (↑)	SR (↑)	SR (↑)	SR (↑)	SR (↑)
Diffusion Policy [14]*	72.4	78.3	92.5	68.3	50.5
Octo [50]*	75.1	78.9	85.7	84.6	51.1
DiT Policy [20]*	82.4	84.2	96.3	85.4	63.8
OpenVLA [28]*	76.5	84.7	88.4	79.2	53.7
π_0 [7]	87.4 $\pm$ 0.2	87.8 $\pm$ 0.2	84.9 $\pm$ 0.4	91.2 $\pm$ 0.9	85.7 $\pm$ 0.7
π_0[7] **+ Pri4R**	90.6 $\pm$ 0.2	92.8 $\pm$ 0.7	88.6 $\pm$ 0.3	95.3 $\pm$ 0.4	85.6 $\pm$ 0.2
$\pi_{0.5}$ [24]	92.6 $\pm$ 0.4	96.1 $\pm$ 0.8	88.3 $\pm$ 0.8	95.6 $\pm$ 0.5	90.5 $\pm$ 1.2
$\pi_{0.5}$ [24] **+ Pri4R**	94.0 $\pm$ 0.2	**97.2** $\pm$ 0.6	88.9 $\pm$ 0.6	95.6 $\pm$ 0.5	94.3 $\pm$ 0.6
OpenVLA-OFT [29]	92.7 $\pm$ 0.1	90.8 $\pm$ 0.3	98.2 $\pm$ 0.0	96.4 $\pm$ 0.4	85.5 $\pm$ 0.2
OpenVLA-OFT [29] **+ Pri4R**	**96.3** $\pm$ 0.2	93.2 $\pm$ 0.4	**98.6** $\pm$ 0.0	**98.1** $\pm$ 0.5	**95.3** $\pm$ 0.3

predicts future 3D displacements $\widehat{\Delta P}_{t:t+H}$ in parallel with the action prediction $\mathbf{a}_{t:t+H}$. This auxiliary branch is temporally aligned with the action horizon and utilized exclusively during training. By discarding the point track head after training, the final policy architecture remains identical to the original VLA, ensuring no additional computational overhead or input requirements at inference time.

D. Training

Data construction. We construct 3D point tracks for both simulation and real world. In simulation, we can access the ground-truth scene mesh from the simulator [52]. Therefore, we initialize N_p query points only at the first frame by cropping the mesh within a robot-centered 3D cube and sampling points over mesh faces. To track these points, we store the corresponding face indices and barycentric coordinates at initialization, and roll out each action sequence while retrieving the 3D locations of the same surface points at every timestep, yielding $\{P_\tau\}_{\tau=1}^{T}$.

For real scenes, we use an off-the-shelf 3D point tracking model [61] to annotate pseudo-labeled tracks. To focus sampling toward foreground regions, we use a segmentation model to sample more points on robots and objects, while sampling background points uniformly. Since our real-world dataset is recorded with a fixed camera setup, we can recover stable world coordinates for the tracked points over time.

Objectives. We follow the original action training objective of each VLA, ℓ_1 regression for OpenVLA-OFT and flow matching for π models. We add an auxiliary ℓ_1 loss on 3D point track displacements $\widehat{\Delta P}_{t:t+H}$. The total loss is

$$\mathcal{L} = \mathcal{L}_{\text{act}} + \omega_{\text{pt}} \left\| \widehat{\Delta P}_{t:t+H} - \Delta P_{t:t+H} \right\|_1.$$

where ω_{pt} controls the balance between two loss terms.

V. Experiments

We evaluate Pri4R's ability to inject knowledge about world dynamics during fine-tuning by using 3D point tracks as privileged supervision. Specifically, we seek to answer the following questions:

1) Does Pri4R improve state-of-the-art VLAs across diverse robot embodiments and tasks?
2) Which characteristics of 3D point tracks are critical for world-dynamics learning and downstream control performance?
3) How do Pri4R design choices affect the performance of the fine-tuned policy?
4) Does Pri4R improve performance on real world tasks that require awareness about world dynamics?

A. Experimental Setup

Baselines. We compare Pri4R against recent VLA and imitation-learning baselines with publicly available code: Diffusion Policy [14], Octo [50], DiT Policy [20], OpenVLA [28], OpenVLA-OFT [29], π_0 [7], and $\pi_{0.5}$ [24]. For Pri4R, we use OpenVLA-OFT, π_0, and $\pi_{0.5}$ as backbones, as they (i) achieve SOTA results across diverse benchmarks and (ii) span two representative VLA architectures: backbone-centric policies with action heads and expert-style action models.

Implementation details. For OpenVLA-OFT [29], we apply LoRA adapters to the pretrained backbone and jointly train the point track head. We set the point supervision weight to $w_{\text{pt}} = 1$ and sample $N_p = 1024$ points per trajectory. All remaining model hyperparameters follow the OpenVLA-OFT default configuration. For π_0 and $\pi_{0.5}$ [7, 24], we use the official PyTorch implementation and fully fine-tune the base model together with the additional point track embedding module and our point track head. We set $w_{\text{pt}} = 1$ and use $N_p = 1024$. All other model hyperparameters follow the default settings used for the corresponding π models.

B. Mutitask Simulation Benchmarks

Experimental setup. We evaluate Pri4R on two multi-task simulation benchmarks: LIBERO [36] and RoboCasa [40]. LIBERO is a language-conditioned tabletop manipulation benchmark that tests multitask generalization across task families with varying spatial layouts, target goals, and object configurations; we report results on four suites (Spatial, Object, Goal, and Long), each with 10 tasks and 50 demonstrations per task. RoboCasa is a large-scale kitchen manipulation benchmark with diverse scenes and articulated interactions; we evaluate 24 atomic tasks using the Human-50 dataset, which provides 50 demonstrations per task.

TABLE II: **Results on Robocasa.** Success rate (SR) for each method across 24 task suites, grouped following [40], and averaged over 50 trials. Pri4R improves average success rates of every state-of-the-art VLAs and almost every task in Robocasa.

	Average	Pick and Place	Open/Close doors	Open/Close drawers	Twisting knobs	Turning levers	Pressing buttons	Insertion
	SR (↑)	SR (↑)	SR (↑)	SR (↑)	SR (↑)	SR (↑)	SR (↑)	SR (↑)
π_0 [7]	38.8	24.0	45.0	78.0	29.0	57.3	57.3	0.0
π_0 [7] **+ Pri4R**	42.2 (+3.4)	24.0 (+0.0)	49.0 (+4.0)	84.0 (+6.0)	30.0 (+1.0)	71.3 (+14.0)	59.3 (+2.0)	2.0 (+2.0)
$\pi_{0.5}$ [24]	52.9	**54.3**	51.0	75.0	28.0	79.3	60.0	4.0
$\pi_{0.5}$ [24] **+ Pri4R**	**57.0** (+4.1)	52.0 (-2.3)	**68.5** (+17.5)	**89.0** (+14.0)	**33.0** (+5.0)	**86.7** (+7.4)	54.7 (-5.3)	5.0 (+1.0)
OpenVLA-OFT [29]	33.1	21.8	45.7	59.0	8.0	36.0	56.0	27.0
OpenVLA-OFT [29]**+Pri4R**	46.3 (+13.2)	23.0 (+1.2)	61.7 (+16.0)	80.0 (+21.0)	25.0 (+17.0)	66.7 (+30.7)	**79.3** (+23.3)	**34.0** (+7.0)

TABLE III: **Analysis of supervision.** Average success rates on RoboCasa. 3D point track is the most effective supervision, while using both robot and scene points is crucial.

Method	**SR** ↑	Δ
OpenVLA-OFT [29]	33.1	–
What supervision signal?		
+ Goal point set	33.8	+0.7
+ 2D point track	37.0	+3.9
+ Depth	42.3	+8.3
+ Ours (3D point track)	**46.3**	**+13.2**
What to track?		
+ Only scene points	35.2	+2.1
+ Only robot points	43.8	+10.7
+ Ours (both scene and robot)	**46.3**	**+13.2**

For both baselines, we train a single multitask policy per benchmark. Inputs include language instruction, proprioception and two camera views (thrid-person and wrist) for LIBERO and three views (left, right, wrist) for RoboCasa. For OpenVLA-OFT, we fine-tune with batch size 128 for 90k steps on LIBERO [36], and batch size 64 for 120k steps on RoboCasa [40]. For π series, we finetune with batch size 128 for 30k steps on both benchmark. Additional implementation details are provided in the appendix.

Results. Table I shows that Pri4R improves the average success rate of all strong VLA baselines on LIBERO. The gains are especially pronounced on LIBERO-Long with OpenVLA-OFT, where Pri4R yields a 9.8% absolute improvement, indicating that privileged geometric 4D supervision encourages better modeling of action–world interactions. On the more challenging RoboCasa benchmark, which requires robust execution under randomized scene configurations, Pri4R achieves even larger success rate gains than on LIBERO (Table II), demonstrating that *privileged* point track supervision transfers beyond temporally structured suites and improves generalization under stronger distribution shift. Moreover, Figure 3 illustrates Pri4R's training dynamics: performance improves more slowly during the first ∼20K steps due to the added point track prediction objective, but then increases rapidly, reaching the baseline's peak performance 2.7× faster. This speedup corresponds to approximately 8× H200 GPU-days (about 24 hours on 8 H200s). Finally, we visualize the predicted point tracks in Figure 4. Additional results are shown in the Appendix.

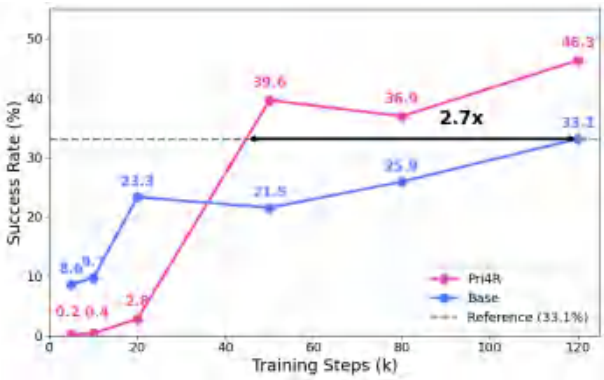

Fig. 3: **Training dynamics.** Pri4R learns slowly at the early stage due to the 3D point track objective, but improves performance rapidly, reaching the baseline peak 2.7× faster.

C. Analysis of 3D Point Track Supervision

Temporality & Spatiality. To isolate the roles of temporal density and metric geometry in point track supervision, we evaluate two controlled variants of our 3D full-horizon displacement target, $\Delta P_{t:t+H}$. For *temporality*, we replace $\Delta P_{t:t+H}$ with a goal-only objective that predicts only the terminal point set P_{t+H}; concretely, we mean-pool backbone embeddings before the fusion MLP and keep the rest unchanged. For *spatiality*, we preserve temporal density but remove metric 3D structure by projecting 3D tracks to a single camera view and supervising with 2D tracks. Table III shows that both goal-only prediction and 2D tracks yield only modest improvements, whereas supervising temporally dense *and* metrically grounded 3D tracks over the full horizon leads to substantially larger gains, indicating that the benefits are strongest when **temporality and spatiality are jointly present** in the supervision signal.

Spatial redundancy To assess whether 3D point tracks provide advantages over spatially dense supervision such as depth, we replace point track prediction with future depth map prediction. Because depth maps are high-resolution, we first train a depth variational autoencoder (VAE) and use its latent codes as supervision instead of raw depth. Table III shows that predicting depth improves performance over the baseline, but it consistently underperforms Pri4R with 3D point tracks. We attribute this gap to two limitations of depth supervision: (i) depth maps are spatially dense and therefore temporally

TABLE IV: **Ablation on P_t as input.** Δ denotes absolute improvement over the OpenVLA-OFT baseline [29]. "P_t input" indicates that the point set P_t is appended to the backbone as additional tokens alongside image and language inputs. "$P_{t:t+H}$ track" refers to our proposed displacement supervision over the action horizon.

	Need P_t at test?	**Avg.**	PnP	Doors	Drawers	Knobs	Lever	Button	Insertion
		SR / Δ	SR / Δ	SR / Δ	SR / Δ	SR / Δ	SR / Δ	SR / Δ	SR / Δ
OpenVLA-OFT [29]	No	33.1 / +0.0	21.8 / +0.0	45.7 / +0.0	59.0 / +0.0	8.0 / +0.0	36.0 / +0.0	56.0 / +0.0	27.0 / +0.0
+ P_t input	Yes	33.3 / +0.2	14.0 / -7.8	48.0 / +2.3	59.0 / +0.0	10.0 / +2.0	49.3 / +13.3	59.3 / +3.3	26.0 / -1.0
+ P_t input + $P_{t:t+H}$ track	Yes	34.5 / +1.4	14.3 / -7.5	49.0 / +3.3	59.0 / +0.0	10.0 / +2.0	54.0 / +18.0	60.7 / +4.7	29.0 / +2.0
+ $P_{t:t+H}$ track (Ours)	**No**	**46.3 / +13.2**	**23.0 / +1.2**	**61.7 / +16.0**	**80.0 / +21.0**	**25.0 / +17.0**	**66.7 / +30.7**	**79.3 / +23.3**	**34.0 / +7.0**

TABLE V: **Ablations on embedding module with $\pi_{0.5}$.** Average success rate (SR) on RoboCasa. Each design choice of our embedding module is important.

Method	**SR** $\uparrow$	Δ
$\pi_{0.5}$ [24]	52.9	–
Design choice of embedding module		
+ Point expert	53.4	+0.5
+ Backbone query token (attend action)	54.4	+1.5
+ Backbone query token	54.8	+1.9
+ Ours	**57.0**	**+4.1**

redundant under a static camera, and (ii) they do not provide identity registration across time. Consequently, depth provides a weaker signal for learning how the scene evolves under interaction compared to tracking consistent 3D points over time.

Scene-robot interaction. Finally, we compare tracking only environment points or only robot points to our world tracking that couples both. Tracking only the robot mainly captures self-motion, while tracking only the scene weakens sensitivity to contact and articulation. Table III shows that neither matches world tracking, suggesting that modeling robot–environment interaction is key to improving performance in complex manipulation tasks.

D. Ablations on Design Choices

Effect of P_t input. A key design choice in Pri4R is to feed the current point set P_t *only to the point track head*, rather than to the backbone [6] or the action head [30, 49]. This has two practical advantages: (1) it requires no 3D observations at inference time, and (2) it mitigates distribution shift from pretraining, where $\mathbf{o}_t$ consists only of images and language. As shown in Table IV, appending P_t to the backbone can improve several suites by injecting explicit 3D cues, but it reduces performance on Pick-and-Place, where language plays a larger role [35]. We attribute this drop to introducing a new token type into the VLM embedding space, which can perturb the pretrained representation [30, 49]. Adding point track supervision on top of the backbone P_t input improves the average success rate, but the PnP degradation persists. In contrast, conditioning *only* the point track head on P_t while supervising it with point tracks yields consistent gains across suites and even improves PnP performance.

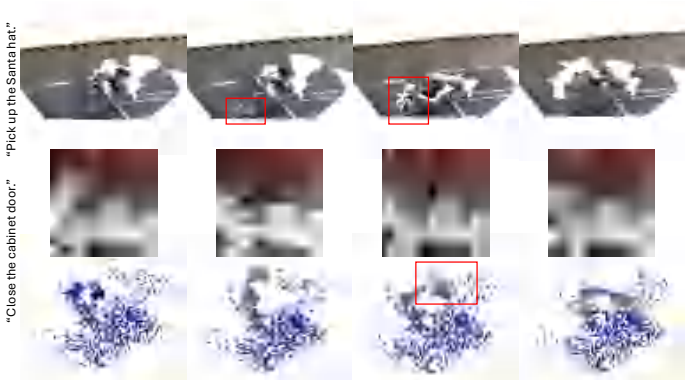

Fig. 4: **Predicted point tracks in simulation and the real world.** Future trajectories are visualized in a rainbow color map. As highlighted in the red boxes, Pri4R accurately predicts point tracks for both scene elements and the robot.

TABLE VI: **Point-track head ablations.** SR of OpenVLA-OFT on LIBERO Long. Our point-track head has two slots: *Point MLP* and *Fusion MLP*. We plug in different architectures for each slot and report SR; Δ is w.r.t. the no-head baseline.

Point Encoder	**Fusion Module**	**SR** $\uparrow$	Δ
OpenVLA-OFT (no point-track head)		89.2	–
PointNet [12]	**Ours**	80.8	-8.4
PtTrans. [60]	**Ours**	92.4	$+3.2$
Ours	Transformer [53]	92.2	$+3.0$
Ours	**Ours**	**94.4**	$\mathbf{+5.2}$

Ablations on the embedding module. While OpenVLA-OFT only requires adding a point track head, the π family additionally needs an embedding module to produce $\mathbf{z}_t \in \mathbb{R}^{H\times d}$. Table V compares several ways of constructing $\mathbf{z}_t$. A simple baseline introduces a *point expert* mirroring the action expert, conditioned via layer-wise self-attention with the backbone. We also evaluate learnable query tokens injected into the backbone [65], either allowing attention to action tokens or masking this interaction. All alternatives underperform our embedding module, highlighting the importance of our design for learning world dynamics via point tracking.

Ablations on point track head. We conduct an ablation study on the tracking head architecture, which consists of two components: (i) a point embedding module and (ii) a fusion module that combines action embeddings with point embeddings to predict point tracks. Table VI compares alternative designs, including PointNet and point-transformer point

TABLE VII: **Quantitative results on the real-world setup.**

Model	Height	Spatial		Depth		Tracking		
	Unseen	Seen	Unseen	Seen	Unseen	Seen	Unseen	OOD
OpenVLA-OFT	83.3	100.0	60.0	45.9	50.0/ 0.0	75.0	41.7	50.0
+ Pri4R	**96.7**	**100.0**	**80.0**	**79.2**	**50.0/37.5**	**100.0**	**66.7**	**66.7**
$\pi_{0.5}$	60.0	90.0	30.0	66.7	50.0/ 0.0	50.0	0.0	33.8
+ Pri4R	**66.7**	**100.0**	**60.0**	**95.8**	**100.0**/0.0	**66.7**	0.0	**50**

embedding, as well as a transformer-based fusion module. Overall, we find that our lightweight MLP design for both modules achieves the best performance, outperforming heavier point encoders and fusion transformers.

E. Real World Evaluation

To evaluate the Pri4R's spatiotemporal capabilities in real world settings, we conduct four real world tasks on the OMY-F3M robot, a one-arm manipulation platform.

Experimental setup OMY-F3M is a one-arm manipulation platform with a 6-DoF arm actuated by DYNAMIXEL-Y motors. We operate the robot at 10Hz with a 7-dimensional action space comprising six target absolute joint angles and one gripper command. Observations are collected from three camera viewpoints (top, wrist-mounted, and left third-person). We finetune OpenVLA-OFT for 60k-150k steps per task with an action chunk size of 10, and execute the full action chunk at inference time. We evaluate models on four real world tasks that require world-dynamics understanding:

1) **Pick-and-place over an obstacle.** The robot must place an object over obstacles of three distinct heights. We collected 45 demonstrations and evaluated 30 trials.
2) **Pick-and-place into a bin.** The robot picks up an object and places it into a bin. The object start location is either seen or unseen during training. We collected 45 demonstrations and evaluated 20 trials.
3) **Pick the farthest object.** The robot must pick up the object farthest from the robot among multiple candidates. Both the target object identity and object positions are randomized. We collected 48 demonstrations and evaluated 40 trials.
4) **Pick a moving object.** The object is relocated while the robot is approaching, requiring the robot to track the moving object and grasp at the updated position. We collected 40 demonstrations and evaluated 36 trials.

Results. Table VII shows the performance on four real world tasks designed to evaluate the understanding of world-dynamics under contact, obstacles, randomized object configurations, reaching with depth-dependent geometry, and target relocation. Pri4R consistently outperforms the OpenVLA-OFT and $\pi_{0.5}$ baselines across all tasks, demonstrating that geometric 4D supervision improves real world spatiotemporal control. As shown in Figure 6, the baseline model tends to collide with obstacles rather than routing around them, approaches incorrect grasp locations when object positions are randomized, and in some cases closes the gripper on empty space yet still executes the remainder of the action chunk as if contact were made. In contrast, Pri4R exhibits stronger spatiotemporal and interaction awareness. It avoids collisions, re localizes targets, grasps objects from unseen locations, maintains geometrically consistent approaches, and executes grasps at the updated object position. This advantage is most evident in the moving-target setting, where Pri4R continues to update the grasp plan as the object relocates, whereas the baseline often stops early at an outdated location, reflecting limited spatiotemporal awareness. Overall, these results suggest that Pri4R transfers spatiotemporal understanding effectively to real world settings and improves robustness under distribution shifts in object placement and dynamics. More details on real world experiments are provided in the appendix.

Fig. 5: **Real-world setup.** Blue boxes indicate the target. In Pick the farthest object, the red box marks a closer distractor object; in Pick up the doll and place in the white bin, the red box marks a distractor.

(a) OpenVLA-OFT [29] (b) **OpenVLA-OFT + Pri4R**

Fig. 6: **Qualitative comparison across tasks.** Baseline (left) failures vs. Our method (right) successes.

VI. Discussion, Limitations, and Future Work

We presented Pri4R, a framework that enhances the world dynamics understanding of VLA models through privileged 4D representations. By supervising the model to predict 3D point tracks during training, we demonstrated that VLA backbones can develop a more physically-aware context, leading to improved control performance without any inference-time overhead. Our results across various benchmarks suggest that capturing the spatiotemporal evolution of a scene is a critical component for robust robot manipulation.

Despite its effectiveness, Pri4R makes no changes at inference time. While this simplicity is a strength, it may also limit performance: additional test-time computation or explicit geometric inputs could further improve robustness. Moreover, we evaluate Pri4R primarily in the fine-tuning setting on demonstration benchmarks and a small set of real-world rollouts, rather than in large-scale pretraining regimes (e.g., Embodiment X). We believe incorporating 3D point-track supervision during pretraining would have an even larger impact on learning action–world dynamics than using it only for fine-tuning. Finally, since Pri4R transfers to real-world settings, and 4D point tracks can be obtained from off-the-shelf tracking models, our approach is directly applicable to large-scale robotics datasets.

Acknowledgments

This work was supported by LG AI Research.

References

[1] Anurag Ajay, Seungwook Han, Yilun Du, Shuang Li, Abhi Gupta, Tommi Jaakkola, Josh Tenenbaum, Leslie Kaelbling, Akash Srivastava, and Pulkit Agrawal. Compositional foundation models for hierarchical planning. *Advances in Neural Information Processing Systems*, 36: 22304–22325, 2023.

[2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 35: 23716–23736, 2022.

[3] Brenna Argall, Sonia Chernova, Manuela Veloso, and Brett Browning. A survey of robot learning from demonstration. *Robotics and Autonomous Systems*, 57(5):469–483, 2009. ISSN 0921-8890.

[4] Suneel Belkhale and Dorsa Sadigh. Minivla: A better vla with a smaller footprint, 2024. URL https://github.com/Stanford-ILIAD/openvla-mini.

[5] Homanga Bharadhwaj, Debidatta Dwibedi, Abhinav Gupta, Shubham Tulsiani, Carl Doersch, Ted Xiao, Dhruv Shah, Fei Xia, Dorsa Sadigh, and Sean Kirmani. Gen2act: Human video generation in novel scenarios enables generalizable robot manipulation. *arXiv preprint arXiv:2409.16283*, 2024.

[6] Vineet Bhat, Yu-Hsiang Lan, Prashanth Krishnamurthy, Ramesh Karri, and Farshad Khorrami. 3d cavla: Leveraging depth and 3d context to generalize vision language action models for unseen tasks. *arXiv preprint arXiv:2505.05800*, 2025.

[7] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[8] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.

[9] Qingwen Bu, Jia Zeng, Li Chen, Yanchao Yang, Guyue Zhou, Junchi Yan, Ping Luo, Heming Cui, Yi Ma, and Hongyang Li. Closed-loop visuomotor control with generative expectation for robotic manipulation. *Advances in Neural Information Processing Systems*, 37:139002–139029, 2024.

[10] Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Shenyuan Gao, Xindong He, Xuan Hu, Xu Huang, et al. Agibot world colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025.

[11] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, et al. Worldvla: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.

[12] R. Qi Charles, Hao Su, Mo Kaichun, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 77–85, 2017.

[13] Xi Chen, Josip Djolonga, Piotr Padlewski, Basil Mustafa, Soravit Changpinyo, Jialin Wu, Carlos Riquelme Ruiz, Sebastian Goodman, Xiao Wang, Yi Tay, et al. Pali-x: On scaling up a multilingual vision and language model. *arXiv preprint arXiv:2305.18565*, 2023.

[14] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.

[15] Michel Desmurget and Scott Grafton. Forward modeling allows feedback control for fast reaching movements. *Trends in Cognitive Sciences*, 4(11):423–431, 2000. ISSN 1364-6613.

[16] Yilun Du, Sherry Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Josh Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. *Advances in neural information processing systems*, 36:9156–9172, 2023.

[17] Haiwen Feng, Junyi Zhang, Qianqian Wang, Yufei Ye, Pengcheng Yu, Michael J Black, Trevor Darrell, and Angjoo Kanazawa. St4rtrack: Simultaneous 4d reconstruction and tracking in the world. *arXiv preprint arXiv:2504.13152*, 2025.

[18] Jan Feyereisl, Suha Kwak, Jeany Son, and Bohyung Han. Object localization based on structural svm using privileged information. In *Advances in Neural Information Processing Systems*, pages 208–216, 2014.

[19] J. Randall Flanagan and Alan M. Wing. The role of internal models in motion planning and control: Evidence from grip force adjustments during movements of hand-held loads. *Journal of Neuroscience*, 17(4):1519–1528, 1997. ISSN 0270-6474.

[20] Zhi Hou, Tianyi Zhang, Yuwen Xiong, Hengjun Pu, Chengyang Zhao, Ronglei Tong, Yu Qiao, Jifeng Dai, and Yuntao Chen. Diffusion transformer policy. *arXiv preprint arXiv:2410.15959*, 2024.

[21] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. Video prediction policy: A generalist robot policy with predictive visual representations. *arXiv preprint arXiv:2412.14803*, 2024.

[22] Jiangyong Huang, Silong Yong, Xiaojian Ma, Xiongkun Linghu, Puhao Li, Yan Wang, Qing Li, Song-Chun Zhu, Baoxiong Jia, and Siyuan Huang. An embodied generalist agent in 3d world. *arXiv preprint arXiv:2311.12871*, 2023.

[23] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. Pointworld: Scaling 3d world models for in-the-wild robotic manipulation. *arXiv preprint arXiv:2601.03782*, 2026.

[24] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[25] Roland Johansson. Sensory input and control of grip. *Novartis Foundation symposium*, 218:45–59; discussion 59, 02 1998. doi: 10.1002/9780470515563.ch4.

[26] Siddharth Karamcheti, Suraj Nair, Ashwin Balakrishna, Percy Liang, Thomas Kollar, and Dorsa Sadigh. Prismatic vlms: Investigating the design space of visually-conditioned language models. In *Forty-first International Conference on Machine Learning*, 2024.

[27] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.

[28] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[29] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[30] Chengmeng Li, Junjie Wen, Yan Peng, Yaxin Peng, Feifei Feng, and Yichen Zhu. Pointvla: Injecting the 3d world into vision-language-action models. *arXiv preprint arXiv:2503.07511*, 2025.

[31] Fuhao Li, Wenxuan Song, Han Zhao, Jingbo Wang, Pengxiang Ding, Donglin Wang, Long Zeng, and Haoang Li. Spatial forcing: Implicit spatial representation alignment for vision-language-action model, 2025. URL https://arxiv.org/abs/2510.12276.

[32] Peiyan Li, Yixiang Chen, Hongtao Wu, Xiao Ma, Xiangnan Wu, Yan Huang, Liang Wang, Tao Kong, and Tieniu Tan. Bridgevla: Input-output alignment for efficient 3d manipulation learning with vision-language models. *arXiv preprint arXiv:2506.07961*, 2025.

[33] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024.

[34] Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. Vision-language foundation models as effective robot imitators. *arXiv preprint arXiv:2311.01378*, 2023.

[35] Shijie Lian, Bin Yu, Xiaopeng Lin, Laurence T. Yang, Zhaolong Shen, Changti Wu, Yuzhuo Miao, Cong Huang, and Kai Chen. Langforce: Bayesian decomposition of vision language action models via latent action queries, 2026.

[36] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.

[37] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.

[38] Jiaming Liu, Hao Chen, Pengju An, Zhuoyang Liu, Renrui Zhang, Chenyang Gu, Xiaoqi Li, Ziyu Guo, Sixiang Chen, Mengzhen Liu, et al. Hybridvla: Collaborative diffusion and autoregression in a unified vision-language-action model. *arXiv preprint arXiv:2503.10631*, 2025.

[39] R.C. Miall and D.M. Wolpert. Forward models for physiological motor control. *Neural Networks*, 9(8):1265–1279, 1996. ISSN 0893-6080. Four Major Hypotheses in Neuroscience.

[40] Soroush Nasiriany, Abhiram Maddukuri, Lance Zhang, Adeet Parikh, Aaron Lo, Abhishek Joshi, Ajay Mandlekar, and Yuke Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. *arXiv preprint arXiv:2406.02523*, 2024.

[41] Dantong Niu, Yuvan Sharma, Haoru Xue, Giscard Biamby, Junyi Zhang, Ziteng Ji, Trevor Darrell, and Roei Herzig. Pre-training auto-regressive robotic models with 4d representations. *arXiv preprint arXiv:2502.13142*, 2025.

[42] Abby O'Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.

[43] Shivansh Patel, Xinchen Yin, Wenlong Huang, Shubham Garg, Hooshang Nayyeri, Li Fei-Fei, Svetlana Lazebnik, and Yunzhu Li. A real-to-sim-to-real approach to robotic manipulation with vlm-generated iterative keypoint rewards. *arXiv preprint arXiv:2502.08643*, 2025.

[44] Dean A Pomerleau. Alvinn: An autonomous land vehicle in a neural network. *Advances in neural information processing systems*, 1, 1988.

[45] Daniel Seita, Yufei Wang, Sarthak J Shetty, Edward Yao Li, Zackory Erickson, and David Held. Toolflownet: Robotic manipulation with tools via predicting tool flow from point clouds. In *Conference on Robot Learning*, pages 1038–1049. PMLR, 2023.

[46] Yichao Shen, Fangyun Wei, Zhiying Du, Yaobo Liang, Yan Lu, Jiaolong Yang, Nanning Zheng, and Baining Guo. Videovla: Video generators can be generalizable robot manipulators. 2025.

[47] Junyao Shi, Joshua Smith, Jianing Qian, and Dinesh Jayaraman. Points2reward: Robotic manipulation rewards from just one video.

[48] Lucy Xiaoyang Shi, Brian Ichter, Michael Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. *arXiv preprint arXiv:2502.19417*, 2025.

[49] Lin Sun, Bin Xie, Yingfei Liu, Hao Shi, Tiancai Wang, and Jiale Cao. Geovla: Empowering 3d representations in vision-language-action models. *arXiv preprint arXiv:2508.09071*, 2025.

[50] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.

[51] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. *arXiv preprint arXiv:2412.15109*, 2024.

[52] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012.

[53] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

[54] Homer Rich Walke, Kevin Black, Tony Z Zhao, Quan Vuong, Chongyi Zheng, Philippe Hansen-Estruch, Andre Wang He, Vivek Myers, Moo Jin Kim, Max Du, et al. Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning*, pages 1723–1736. PMLR, 2023.

[55] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer, 2025.

[56] Chuan Wen, Xingyu Lin, John So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. *arXiv preprint arXiv:2401.00025*, 2023.

[57] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.

[58] Daniel M. Wolpert, Zoubin Ghahramani, and Michael I. Jordan. An internal model for sensorimotor integration. *Science*, 269(5232):1880–1882, 1995.

[59] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. *arXiv preprint arXiv:2312.13139*, 2023.

[60] Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point transformer v3: Simpler, faster, stronger. In *CVPR*, 2024.

[61] Yuxi Xiao, Jianyuan Wang, Nan Xue, Nikita Karaev, Iurii Makarov, Bingyi Kang, Xin Zhu, Hujun Bao, Yujun Shen, and Xiaowei Zhou. Spatialtrackerv2: 3d point tracking made easy. In *ICCV*, 2025.

[62] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. *arXiv preprint arXiv:2407.15208*, 2024.

[63] Zhao-Heng Yin, Sherry Yang, and Pieter Abbeel. Object-centric 3d motion field for robot learning from human videos. *arXiv preprint arXiv:2506.04227*, 2025.

[64] Wentao Yuan, Jiafei Duan, Valts Blukis, Wilbert Pumacay, Ranjay Krishna, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. Robopoint: A vision-language model for spatial affordance prediction for robotics. *arXiv preprint arXiv:2406.10721*, 2024.

[65] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, XinQiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, et al. Dreamvla: A vision-language-action model dreamed with comprehen-

sive world knowledge. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[66] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1702–1713, 2025.

[67] Haoyu Zhen, Xiaowen Qiu, Peihao Chen, Jincheng Yang, Xin Yan, Yilun Du, Yining Hong, and Chuang Gan. 3d-vla: A 3d vision-language-action generative world model. *arXiv preprint arXiv:2403.09631*, 2024.

[68] Ruijie Zheng, Yongyuan Liang, Shuaiyi Huang, Jianfeng Gao, Hal Daumé III, Andrey Kolobov, Furong Huang, and Jianwei Yang. Tracevla: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. *arXiv preprint arXiv:2412.10345*, 2024.

[69] Zhongyi Zhou, Yichen Zhu, Minjie Zhu, Junjie Wen, Ning Liu, Zhiyuan Xu, Weibin Meng, Yaxin Peng, Chaomin Shen, Feifei Feng, et al. Chatvla: Unified multimodal understanding and robot control with vision-language-action model. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 5377–5395, 2025.

[70] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.

In this Appendix, we additionally provide,

- **S.I. Real World Experiment Details**
  - A. Real World Setup
  - B. Real World Experiments Details
  - C. Hyperparameters and Training Details
- **S.II. Simulation Experiment Details**
  - A. Implementation Details for Depth Map Prediction
  - B. Robocasa Full Benchmark Results
  - C. Hyperparameters and Training Details
- **S.III. Additional Experiments**
  - A. Additional Analysis on P_t
  - B. Comparison to VLAs with implicit 3D awareness
  - C. Additional Ablations on $\pi_{0.5}$
- **S.IV. Additional Qualitative Results**
  - A. 3D Point Track Visualization
  - B. Real world rollout images

Moreover, we provide real world experiment videos in the supplementary materials.

Appendix A
Real World Experiment Details

A. Real World Setup

Our robot system overview is illustrated in Figure S1. We use an OMY robot equipped with a 6-DoF manipulator driven by high precision DYNAMIXEL-Y actuators, supporting payloads up to 3kg, along with a 1-DoF gripper. For perception, we mount an Intel RealSense D405 wrist camera and use Intel RealSense D435 cameras for third-person and top-down views.

B. Real World Experiment Details

We conduct four real world manipulation tasks to evaluate the model's spatiotemporal understanding and control. In the main paper Table VII and appendix Table **??**, the columns **Height, Spatial, Depth, and Tracking** correspond to the four tasks described below.

Pick-and-place over an obstacle (Height) The instruction is *"Pick up the doll and place it on the right"*. The robot must pick up a doll and place it to the right side of the workspace while avoiding an obstacle in between. We evaluate three obstacle settings: no obstacle, mid-height obstacle, and high obstacle (twice the mid-height). We collect 45 demonstrations and evaluate 10 trials for each setting (30 total).

Pick-and-place into a bin (Spatial) The instruction is *"Pick up the doll and put it in the white bin"*. We evaluate two conditions. Seen uses in-distribution object placements without distractors. Unseen includes either (i) an additional distractor object at a seen placement or (ii) a target object placed at an unseen location. We collect 45 demonstrations and evaluate 10 trials for Seen and 10 trials for Unseen.

Pick the farthest object (Depth) The instruction is *"Pick up the hat farthest from the robot"*. Two hats are placed on the table, and the robot must select and grasp the one with the larger distance from the robot base. The target identity is balanced across trials (each hat is the target in half of the trials). For Seen, both hats are placed at in-distribution locations. For Unseen, we report results for two sub-settings indicated in the table as "a/b": a denotes the case where one of the two hats is at an unseen location, and b denotes the case where both hats are at unseen locations. We collect 48 demonstrations and evaluate 24 trials for Seen and 16 trials for Unseen.

Pick a moving object (Tracking) The instruction is *"Pick up the Santa hat"*. The target hat is relocated while the robot is approaching, requiring the robot to track the target and execute the grasp at the updated position. Seen uses in-distribution initial and relocated positions. Unseen uses an unseen relocated position (initial position remains in-distribution). OOD additionally relocates the hat twice, inducing stronger distribution shift in the target's motion. Our dataset includes sequences with a fixed target and sequences with a single relocation. We collect 40 demonstrations and evaluate 12 trials each for Seen, Unseen, and OOD (total 36).

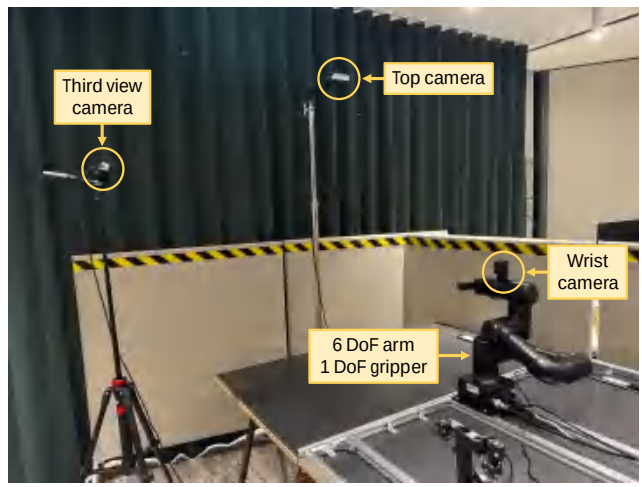

Fig. S1: **Realworld setup**

C. Hyperparameters and Training Details

Table S1 and Table S2 report the hyperparameters used in our real-world training. We follow a shared training protocol across tasks (Height/Spatial/Depth/Tracking) and methods, and modify only the tracking loss weight for Pri4R.

Appendix B
Simulation Experiment Details

A. Implementation Details for Depth Map Prediction

In Table III, we conduct experiments with alternative supervision targets to analyze the benefit of 3D point tracks. Specifically, to compare against spatially dense depth supervision, we predict future depth maps using a compact latent target. Because raw depth maps are high-resolution, we first train a depth variational autoencoder (VAE) and use its latent codes as supervision. This compresses the representation from

TABLE S1: **Hyperparameter details for real-world training.** Unless otherwise specified, OpenVLA-OFT and OpenVLA-OFT+Pri4R share the same settings.

	OpenVLA-OFT	OpenVLA-OFT + Pri4R
Shared settings		
# GPUs	4 × NVIDIA H200	
learning rate (LR)	5e−4	
total batch size	32 (8 per GPU)	
input images	3 camera images: third-person, wrist, top-down	
input image size	224 × 224	
LoRA rank	32	
action chunk size	10	
use proprio (robot state)	yes	
use FiLM	no	
Task-specific training steps		
Height	60K	
Spatial	150K	
Depth	70K	
Tracking	80K	
Method-specific setting		
point track loss weight (ω_{pt})	–	1.0

TABLE S2: **Hyperparameter details for training π series on real-world tasks.** Unless otherwise specified, π and π+Pri4R share the same settings.

	$\pi_{0.5}$	$\pi_{0.5}$ **+ Pri4R**
Shared settings		
# GPUs	4 × NVIDIA H200	
optimizer	AdamW	
input images	3 camera images: third-person, wrist, top-down	
input image size	224 × 224	
learning rate scheduler	Cosine decay	
warmup steps	10,000	
learning rate (LR)	5e−5	
total batch size	32 (8 per GPU)	
EMA decay	0.999	
Task-specific training steps		
Height	40K	
Spatial	30K	
Depth	40K	
Tracking	30K	
Method-specific setting		
point track loss weight (ω_{pt})	–	1.0

256×256 to $32 \times 32 \times 4$ (4096 dimensions), which is comparable to the dimensionality of our 3D point-track representation. Figure S2 visualizes the depth-VAE reconstructions.

TABLE S3: **Simulation training details.** We report settings that differ from real-world training.

Dataset	Method	Input images	# GPUs	Batch size	# steps
LIBERO	OpenVLA-OFT	third-view + wrist-view	8	total 64	90K
	π_0 / $\pi_{0.5}$	third-view + wrist-view	4	total 128	30K
RoboCasa	OpenVLA-OFT	left-view + right-view + wrist-view	8	total 64	120K
	π_0 / $\pi_{0.5}$	left-view + right-view + wrist-view	4	total 128	30K

B. Robocasa Full Benchmark Results

We show the full results on Robocasa on Table S4.

C. Hyperparameters and Training Details

We follow the same hyperparameter and training protocol as in our real world experiments (Table S1 and Table S2) unless otherwise specified. Table S3 summarizes the simulation-specific differences, including camera inputs, compute budget, and training steps for LIBERO and RoboCasa.

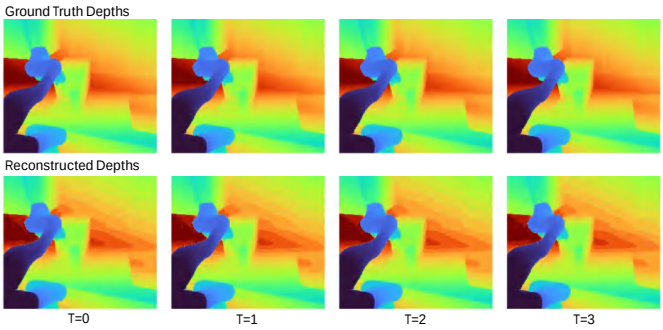

Fig. S2: **Depth reconstruction: (Top)** ground-truth depth maps from the simulator, and **(Bottom)** depth maps reconstructed by our variational autoencoder (VAE).

Appendix C
Additional Experiments

A. Additional Analysis on input P_t

Extending Table IV, we study a variant that removes the current point set P_t entirely, from both the point track head input and the backbone. This variant still requires no 3D observations at inference time, consistent with Pri4R. However, without P_t, the point track head must *generate* future point tracks rather than predict *how a given scene evolves over time*. As shown in Table S5, predicting 3D point tracks without P_t degrades success and can underperform the baseline. This highlights that conditioning the point track head on P_t is crucial for learning world dynamics and, in turn, improving performance.

B. Comparison with VLAs with implicit 3D awareness

SpatialForcing. SpatialForcing [31] recently proposed a complementary approach for improving 3D awareness in VLAs by aligning intermediate VLA features with representations from a 3D geometric foundation model, VGGT [55]. Similar to Pri4R, this does not require explicit 3D observations as additional inputs at test time, while also endowing VLAs with additional knowledge.

Table S6 compares SpatialForcing and Pri4R on OpenVLA-OFT. While SpatialForcing yields consistent gains over the baseline, Pri4R achieves higher success rates. We attribute this gap to the nature of the supervision: SpatialForcing primarily injects static 3D structure from an off-the-shelf model, whereas Pri4R leverages temporally dense 4D supervision through 3D point tracks, explicitly encouraging the policy to model interaction dynamics.

C. Additional Ablations

Ablation on ω_{pt}. Table S7 ablates the weight of the auxiliary point track loss, ω_{pt}. We find that $\omega_{\text{pt}} = 1.0$ performs best among the tested values. Notably, because our point track supervision is formulated as displacements (analogous to the

TABLE S4: **Robocasa success rates by task.** We report the average success rate (**Total**) and per-task success rates on RoboCasa for all experiments.

Method	Total	PnP	Doors	Drawers	Knobs	Lever	Press	Insert
OpenVLA-OFT	0.331	0.218	0.457	0.590	0.080	0.360	0.560	0.270
OpenVLA-OFT + 2D tracks	0.370	0.210	0.520	0.620	0.140	0.473	0.600	0.290
OpenVLA-OFT + Goal point set	0.338	0.165	0.533	0.810	0.080	0.413	0.587	0.220
OpenVLA-OFT + Environment points	0.352	0.190	0.487	0.470	0.170	0.540	0.493	0.280
OpenVLA-OFT + Robot-only points	0.438	0.223	0.607	0.790	0.280	0.660	0.673	0.270
OpenVLA-OFT + **Ours (Pri4R)**	**0.463**	**0.230**	**0.617**	**0.800**	**0.250**	**0.667**	**0.793**	**0.340**
$\pi_{0.5}$	0.529	0.543	0.510	0.750	0.280	0.793	**0.600**	0.040
$\pi_{0.5}$ + backbone query	0.544	**0.553**	0.605	0.750	0.330	0.827	0.507	0.030
$\pi_{0.5}$ + query attend action	0.548	0.535	0.605	0.840	**0.350**	0.767	0.560	0.050
$\pi_{0.5}$ + **Ours (Pri4R)**	**0.570**	0.520	**0.685**	**0.890**	0.330	**0.867**	0.547	**0.050**

TABLE S5: **Analysis of Point Set P_t input.** We present the average success rates on RoboCasa.

Method	SR ↑	Δ
OpenVLA-OFT [29]	33.1	–
+ 3D point sequence w/o input pointcloud	28.7	-4.4
+ Ours (3D point track)	**46.3**	**+13.2**

TABLE S6: **Comparison with SpatialForcing.** We compare Pri4R with a recent SOTA method, SpatialForcing.

	Average	Spatial	Object	Goal	Long
OpenVLA-OFT	92.7	92.8	98.4	96.4	83.0
+SpatialForcing	94.2	**96.8**	**99.0**	96.2	84.8
+Pri4R	**95.0**	93.2	96.6	**96.8**	**93.2**

action displacement targets), the Pri4R is not highly sensitive to ω_{pt} and requires minimal tuning.

Ablation on N_p. Table S8 ablates the number of tracked points, N_p. Although 3D point tracks are intentionally sparse to serve as an efficient supervision signal, using too few points (e.g., 256 or 512) degrades performance compared to our default choice of 1024 points. This suggests that a moderate point density is necessary to sufficiently capture interaction-relevant geometry.

Appendix D
Additional Qualitative Results

A. 3D point track visualization

In Fig. S3, we visualize the 3D point track data extracted from simulation.

B. Real world rollout images

In Fig. S4, we show Pri4R performing real world tasks. Additional qualitative results are provided in the supplementary videos.

TABLE S7: **Ablations on tracking loss weight for $\pi_{0.5}$.** We present the average success rate (SR) across 24 task suites on RoboCasa, averaged over 50 trials per suite.

weight	Avg	PnP	Doors	Drawers	Turn	Twist	Press	Insert
0.1	54.7	54.3	62.5	90.0	28.0	73.3	55.3	3.0
1.0	57.0	52.0	68.5	89.0	33.0	86.7	54.7	5.0
10.0	50.7	47.0	54.5	77.0	35.0	82.0	47.3	5.0

TABLE S8: **Ablations on number of points for 3D track prediction for $\pi_{0.5}$.** We present the average success rate (SR) across 24 task suites on RoboCasa, averaged over 50 trials per suite.

number	Avg	PnP	Doors	Drawers	Turn	Twist	Press	Insert
256	48.8	49.3	55.5	77.0	28.0	73.3	41.3	1.0
512	53.9	48.8	60.0	87.0	27.0	81.3	59.3	7.0
1024	57.0	52.0	68.5	89.0	33.0	86.7	54.7	5.0

Fig. S3: **Visualization of point track data.** Purple lines denote 3D point tracks. Points are sub-sampled for clarity.

Fig. S4: **Pri4R on real-world tasks.** Additional qualitative results are provided in the supplementary videos.